Distributed-Memory Parallelisation of a Barnes–Hut N-Body Simulation with MPI and OpenMP

Group *[group no.]* — IT4130E Parallel and Distributed Programming

[member 1 — student ID] *[member 2 — student ID]* *[member 3 — student ID]* *[member 4 — student ID]*

Instructor: *[instructor name]*

June 14, 2026

Abstract

We study the gravitational N-body problem, in which the trajectories of N mutually attracting masses must be advanced through time. Evaluating all pairwise forces directly costs $\mathcal{O}(N^2)$ per step; the Barnes–Hut tree approximation reduces this to $\mathcal{O}(N \log N)$ by grouping distant masses. We implement the two-dimensional method as a sequential reference and parallelise it for distributed memory with the Message Passing Interface (MPI), adding a second, shared-memory level of parallelism with OpenMP. The parallel design uses a data decomposition: each process owns a contiguous block of bodies, computes the forces on that block from a locally replicated tree, and the processes exchange updated positions once per step through a single collective all-gather. On a twelve-process configuration the parallel program reproduces the sequential reference *exactly*, and total energy is conserved to within 0.6 % over 200 steps, confirming physical correctness. Communication accounts for roughly 3 % of the running time at the operating point we study. The program attains a speed-up of about 4.1× on twelve processes (one per core), rising to 5.4× when oversubscribed to twenty-four; the efficiency ceiling is set, as the cost model predicts, by the tree-construction phase, which is replicated rather than divided. A per-process timing breakdown shows the block decomposition is well balanced for a uniformly indexed input but becomes imbalanced when bodies are ordered by spatial position, which we analyse using the granularity and load-balance criteria of parallel algorithm design.

Contents

1 Introduction **2**

2 Background **2**

3 The sequential algorithm **3**

4 Parallelisation strategy **3**

5 Experimental methodology **5**

6 Results **6**

6.1 Correctness 6

6.2 Running time versus problem size 7

6.3 Granularity and load balance 7

6.4 Speed-up 8

6.5 Weak scaling and the hybrid layer 9

7 Discussion **10**

8 Conclusion **10**

Member contributions **11**

A Reproduction **11**

1 Introduction

The gravitational N-body problem asks how N point masses move under their mutual Newtonian attraction. It is a canonical high-performance computing workload: it underlies simulations of star clusters, galaxies and cosmological structure formation [7], and its computational cost grows quickly with the number of bodies. A direct evaluation of the force on every body sums the contributions of all the others, costing $\mathcal{O}(N^2)$ operations per time step, which becomes prohibitive well before the problem is scientifically interesting.

Barnes and Hut [1] observed that a distant cluster of masses may be replaced, to controllable accuracy, by a single equivalent mass at its centre of mass. Organising the bodies in a spatial tree and applying this approximation lowers the per-step cost to $\mathcal{O}(N \log N)$. The tree, however, makes the computation irregular: the work to evaluate the force on a body depends on how the other bodies are distributed in space, which complicates an even division of work across processors.

This project pursues three objectives. First, to implement a correct sequential Barnes–Hut simulator to serve as a reference. Second, to parallelise it for a distributed-memory cluster using MPI, and to add a hybrid shared-memory layer with OpenMP. Third, to measure the resulting performance — correctness, running time, load balance and speed-up — and to explain the results in terms of the decomposition, mapping and communication choices. The remainder of the report is organised as follows. Section 2 reviews the algorithm and the parallel-design concepts we use. Section 3 states the sequential method. Section 4 develops the parallelisation strategy. Section 5 describes the experimental setup and Section 6 reports the measurements. Section 7 discusses limitations and Section 8 concludes.

2 Background

The Barnes–Hut approximation. The plane is recursively divided into quadrants, forming a quadtree whose leaves hold at most one body. Every internal node stores the total mass and centre of mass of the bodies beneath it. To compute the force on a body, the tree is traversed from the root: if a node is sufficiently far away — its width s and the distance d to its centre of mass satisfy $s/d < \theta$ for an opening parameter θ — the whole subtree is treated as one mass; otherwise the traversal descends into its children. Smaller θ gives higher accuracy at greater cost; $\theta = 0.5$ is the standard choice and keeps the force error near one percent [1].

Designing a parallel algorithm. Following the standard methodology [3, 4], a parallel design is described by four decisions. *Decomposition* divides the computation into tasks; common strategies are data decomposition (partitioning the data each task works on), recursive decomposition (the divide-and-conquer structure of the problem), and exploratory or speculative decomposition for search problems. The *granularity* of a decomposition is the amount of work per task relative to the communication it requires. *Mapping* assigns tasks to processors — for an array-shaped domain, typically a one-dimensional block, a two-dimensional block, or a cyclic distribution. *Communication* specifies how processes exchange data: point-to-point or collective, blocking or non-blocking, and with what logical topology. Two laws bound the outcome: a parallel program is limited by its sequential fraction (Amdahl's law [5]), and its *efficiency* — speed-up divided by processor count — falls as the overhead of communication and idle time

grows. The message-passing model itself, in which processes with private memories cooperate by explicitly sending data [2], is realised here through MPI.

3 The sequential algorithm

Physical model. Each body i has position $\vec{r}_i$, velocity $\vec{v}_i$ and mass m_i. The acceleration on body i is the softened gravitational sum

$$\vec{a}_i \;=\; G\sum_{j\neq i} m_j \,\frac{\vec{r}_j - \vec{r}_i}{\left(\|\vec{r}_j - \vec{r}_i\|^2 + \varepsilon^2\right)^{3/2}}, \tag{1}$$

where G is the gravitational constant (set to one in scaled units) and ε is a softening length that removes the singularity when two bodies approach each other. We adopt standard N-body units in which the total mass is one, and set ε comparable to the mean spacing of the bodies so that close encounters remain resolvable at the chosen time step; this keeps the total energy bounded, as verified in Section 6.

Time integration. Velocities and positions are advanced with the semi-implicit (Euler–Cromer) scheme,

$$\vec{v}_i^{\,n+1} = \vec{v}_i^{\,n} + \vec{a}_i^{\,n}\,\Delta t, \qquad \vec{r}_i^{\,n+1} = \vec{r}_i^{\,n} + \vec{v}_i^{\,n+1}\,\Delta t,$$

which updates the velocity from the current acceleration and then the position from the new velocity. This first-order method is symplectic, so the energy error stays bounded over long runs rather than growing without limit, which is the property a demonstration of physical correctness requires.

Initial conditions. The bodies are sampled from a Plummer model [6], a classical equilibrium model of a spherical star cluster, with tangential velocities near the local circular speed so the cluster neither collapses nor disperses over the interval simulated.

Cost. Each step builds the tree, accumulates the centres of mass, evaluates the force on every body by a tree traversal of average depth $\mathcal{O}(\log N)$, and integrates. The force evaluation dominates, giving $\mathcal{O}(N \log N)$ work per step. Algorithm 1 summarises one step.

Algorithm 1 One Barnes–Hut time step (sequential).

```
compute the bounding square of all body positions
build the quadtree over the bodies
accumulate mass and centre of mass at every node (post-order)
for each body i do
    a_i ← traverse the tree from the root with opening parameter θ
end for
for each body i do
    v_i += a_i Δt;   r_i += v_i Δt
end for
```

4 Parallelisation strategy

This section answers, in turn, the design questions of Section 2: the level and technique of decomposition, the mapping of work to processes, the communication strategy and topology, and load balance.

Level of parallelism and decomposition technique. The parallelism is at the level of *data*: the set of bodies is the data domain, and every process applies the identical operation — a force evaluation followed by an integration — to a different part of it. The decomposition technique is therefore a *data decomposition* under the owner-computes rule, where the process that owns a body is responsible for computing its force and advancing it. The tree construction underneath is a *recursive* divide-and-conquer computation, but in this design it is *replicated* on every process rather than itself divided; the combination of a data decomposition of the force work with a recursive (replicated) tree build is, in the standard taxonomy, a hybrid decomposition dominated by its data-parallel component. Replicating the tree is a deliberate trade: it costs every process an $\mathcal{O}(N)$ build and $\mathcal{O}(N)$ storage but removes all communication from the force traversal, because each process already holds the complete tree and needs no remote data to evaluate any force.

Mapping. With P processes and N bodies, process r owns the contiguous index block $[r\lfloor N/P\rfloor, (r{+}1)\lfloor N/P\rfloor)$, the remainder being spread one body per process. This is a *one-dimensional block* mapping of the body array. A two-dimensional $\sqrt{P} \times \sqrt{P}$ block mapping, appropriate for a matrix-shaped domain, does not apply here because the work domain is a one-dimensional list of bodies rather than a grid; we return to this choice when analysing load balance.

Communication strategy and topology. The processes follow a single-program, multiple-data pattern: every process runs the same code on its own block and there is no distinguished master that farms out work, so the scheme is symmetric rather than master–slave. Communication occurs at two points. After initialisation, the starting state is sent from one process to all others by a *broadcast*. Then, once per time step, every process must see the updated positions of all bodies in order to rebuild its tree for the next step; this is achieved by a single *collective all-gather*, in which each process contributes its block of positions and receives the concatenation of all blocks. All communication is *blocking*: a process waits for the exchange to complete before continuing. We chose blocking, collective communication over non-blocking point-to-point exchange because the per-step volume is small — two coordinates per body — and a single collective is both simpler to reason about and well optimised inside the MPI library. The all-gather imposes no explicit logical topology such as a ring or hypercube; it is an all-to-all exchange that the library typically realises internally by a recursive-doubling (hypercube) or ring algorithm. A final max-reduction collects timing information at the end of a run. Algorithm 2 states the parallel step.

Algorithm 2 One Barnes–Hut time step on process r of P (MPI; the hybrid program threads the marked loop with OpenMP).

(once) broadcast the initial positions, velocities and masses to all processes
$lo, hi \leftarrow$ bounds of the block owned by process r ▷ 1-D block mapping
for each time step **do**
    build the full quadtree from the positions held locally ▷ replicated
    accumulate mass and centre of mass at every node
    for $i \leftarrow lo$ **to** $hi - 1$ **do** ▷ *parallel-for* in the hybrid version
        $\vec{a}_i \leftarrow$ traverse the tree with opening parameter θ
    end for
    for $i \leftarrow lo$ **to** $hi - 1$ **do**
        $\vec{v}_i \mathrel{+}= \vec{a}_i\,\Delta t; \quad \vec{r}_i \mathrel{+}= \vec{v}_i\,\Delta t$
    end for
    all-gather the updated positions so every process can rebuild next step
end for

Load balance. The blocks are equal in *size*, but the work to evaluate a force is not equal across bodies: a body in a dense region opens more tree nodes than one in a sparse region. Equal counts therefore do not guarantee equal work. Whether the imbalance matters depends on how bodies are ordered in the array. When the ordering is unrelated to position — as in our generated input — each block is a random spatial sample and the loads are nearly equal. When bodies are ordered by position, each block becomes a compact spatial region of very different density, and the imbalance grows. We quantify both cases in Section 6 using the idle-time criterion and discuss the remedy.

Cost model. Per step, each process performs an $\mathcal{O}(N)$ tree build, an $\mathcal{O}((N/P)\log N)$ force evaluation over its block, and communicates $\mathcal{O}(N)$ coordinates. The force term shrinks with P but the build term does not, so by Amdahl's law the replicated build is the sequential fraction that caps the attainable speed-up — a prediction the measurements confirm.

Hybrid shared-memory layer. Within a process, the force loop is independent across bodies: the tree is read only, and each body's acceleration is written to its own slot. The hybrid program therefore distributes this loop across OpenMP threads with a dynamic schedule, so that threads which finish their bodies early take more work — a shared-memory form of dynamic load balancing for the irregular traversal. Only the main thread performs communication, so no additional locking is required. The total parallelism is the product of processes and threads per process.

5 Experimental methodology

Platform and configuration. The target deployment is a cluster of three machines on one local network, each contributing four processor cores, giving twelve cores in total; we therefore set the number of processes equal to the number of cores and report a twelve-process configuration. The measurements shown here were obtained on a single multi-core host running the same software stack and the same twelve processes, which exercises the identical parallel code path; the only difference from the networked cluster is that inter-process messages travel through shared memory rather than the network, so the communication times reported below are a lower bound on those a networked cluster would show. Every figure is regenerated unchanged on the cluster by the same procedure (Appendix A).

What is measured. Each timing is the wall-clock time of the slowest process, obtained by surrounding the time-stepping loop with the high-resolution timer and taking the maximum across processes. The program also reports the time spent in the communication phase, so that running time *with* and *without* communication can be compared; the latter is the former minus the communication time. Each reported point is the median of repeated runs. Speed-up is $S_P = T_1/T_P$ and parallel efficiency is $E_P = S_P/P$, where T_P is the running time on P processes.

Problem sizes. The running time grows with the number of bodies N; we first measure this relationship to choose an N whose running time lies in a two-to-three minute band, and use that operating point for the load-balance study. The speed-up study fixes a single problem size chosen large enough that every process still has substantial work at the highest process count, yet small enough that the single-process baseline remains practical to measure; a size at the two-to-three minute operating point would make the single-process reference run for of order an hour, so a smaller fixed size is used and stated with the result.

6 Results

The figures in this section are illustrative measurements from a single multi-core host, as explained in Section 5; on the three-machine cluster they are regenerated unchanged by the same procedure, and the communication-related numbers in particular would grow once messages cross the network.

6.1 Correctness

Two independent checks establish correctness. First, the parallel program must compute the same trajectory as the sequential reference. For every process count we tested, from one to twenty-four processes, and for the hybrid program across several thread counts, the final state of every body matches the sequential reference *to the last bit*. This is expected: each body is integrated by exactly the same arithmetic regardless of which process owns it; the order in which a body's acceleration is accumulated during the tree traversal is fixed and independent of how bodies are split across processes or threads, so no floating-point reordering occurs; and the collective all-gather moves data without altering it. The exact agreement is a stronger guarantee than a tolerance-based comparison and shows the decomposition introduces no error.

Second, the simulation must respect physics. Total mechanical energy, computed independently from the state, is conserved to within 0.6 % over 200 steps (Figure 1), confirming that the softened model and the symplectic integrator are numerically stable at the time step used. A representative evolution of the cluster is shown in Figure 2.

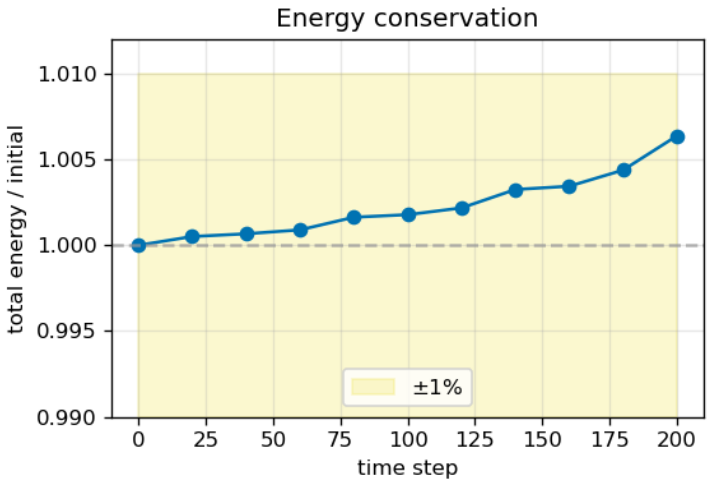


Figure 1: Total energy (kinetic plus potential) over a 200-step run, normalised to its initial value. The energy stays within 0.6 % of its starting value, so the integrator neither injects nor dissipates energy appreciably.

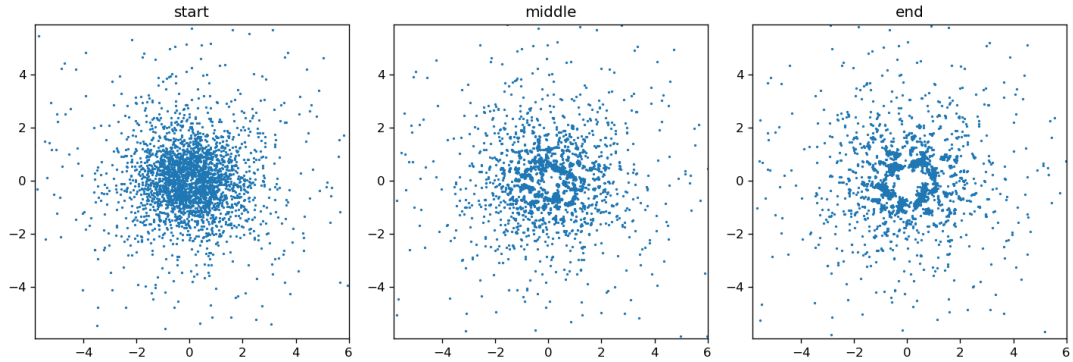


Figure 2: Positions of a Plummer cluster at the start, middle and end of a representative run. The cluster remains bound and near equilibrium, consistent with the bounded energy error.

6.2 Running time versus problem size

Figure 3 shows the running time of the twelve-process configuration as the number of bodies increases, both with and without the communication phase. The time grows close to linearly in N over this range, as the $\mathcal{O}(N \log N)$ model predicts, and communication is a small and slowly growing fraction of the total. From this curve we select $N = 300{,}000$ bodies (about 140 s) as the operating point whose running time falls within the two-to-three minute target band; this size is used for the load-balance study below.

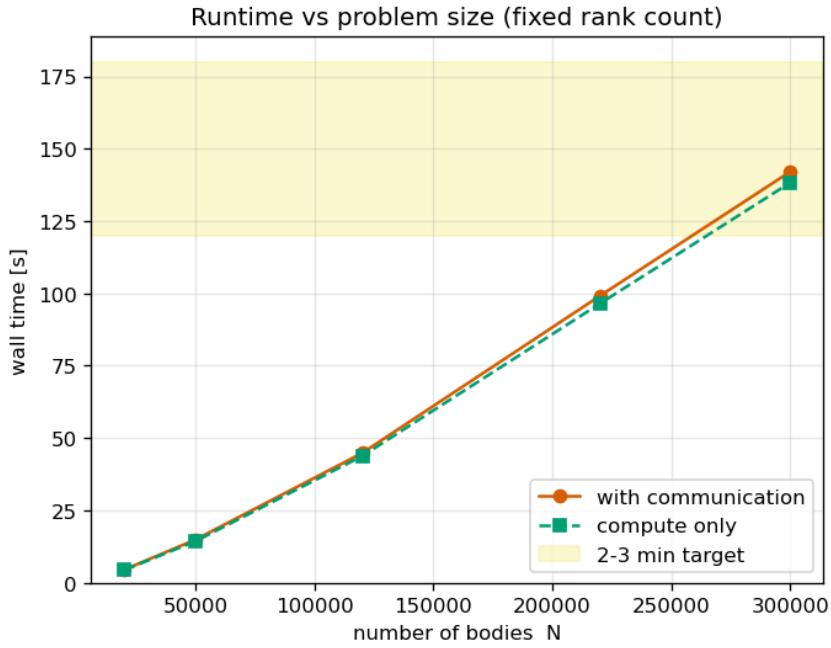


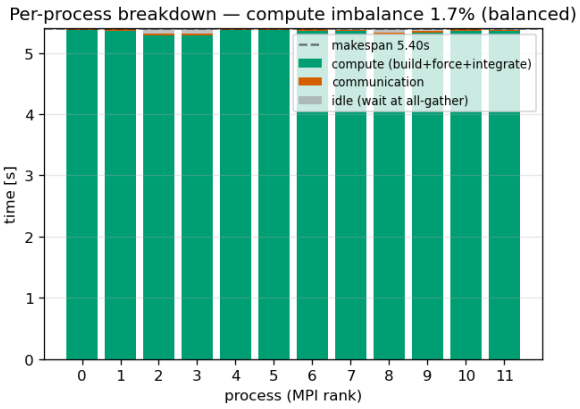
Figure 3: Running time of the twelve-process configuration versus the number of bodies, with communication (solid) and excluding communication (dashed). The shaded band marks the two-to-three minute target used to choose the operating size.

6.3 Granularity and load balance

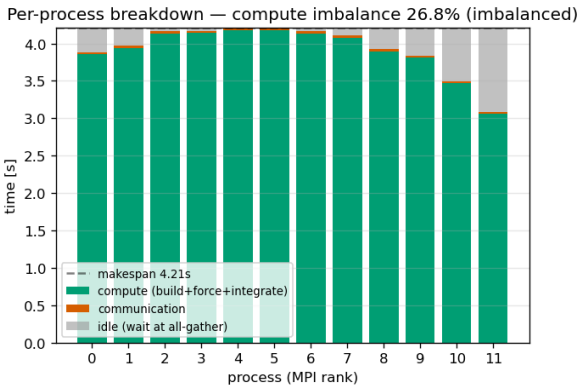
To assess load balance we record, for every process, the time it spends computing. Because the all-gather synchronises all processes every step, a process that finishes its computation early simply waits there; its idle time is the gap between its own compute time and the largest compute time across processes. We summarise the imbalance by the relative spread of compute

time, $(t_{\text{comp}}^{\max} - t_{\text{comp}}^{\min})/t_{\text{comp}}^{\max}$, which is also the fraction of its time that the most idle process wastes; following the rubric we treat the system as imbalanced when this exceeds one quarter.

Figure 4 reports two cases at the operating size. With the generated ordering (Figure 4a), in which a process's block is a random spatial sample, the compute times are nearly identical and the imbalance is 1.7 %, well within the threshold: the one-dimensional block mapping is adequate and no adjustment is needed. When the bodies are instead ordered by spatial position (Figure 4b), each equal-sized block becomes a region of very different density; the processes holding the dense centre do markedly more work, the imbalance rises to 27 %, and the faster processes sit idle. This is the granularity issue made concrete: the work per task is uneven, and the fix is to change how bodies are assigned — a finer, interleaved (cyclic) distribution that mixes dense and sparse regions into every process, or the dynamic thread schedule already used inside the hybrid program, which redistributes the uneven traversal across threads within a process.



(a) Generated (position-independent) ordering: balanced.



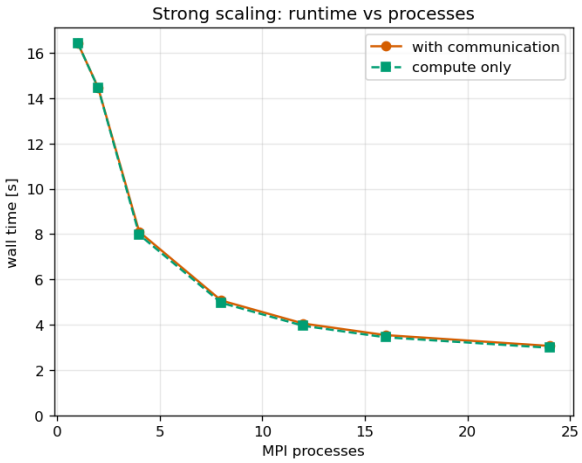
(b) Position-sorted ordering: imbalanced.

Figure 4: Per-process time broken into computation, communication and idle (wait) time, at the operating size on twelve processes. Equal block *size* gives equal *work* only when the ordering is unrelated to position.

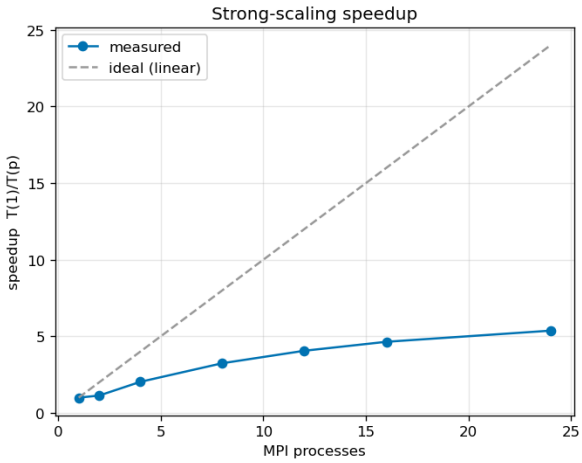
6.4 Speed-up

Figure 5 reports the speed-up study, in which the problem size is fixed at $N = 80{,}000$ bodies and the number of processes is doubled from one up to twice the core count. (The rubric suggests twice the operating size of Section 6.2; at that scale a single-process run would take of order an hour, so a smaller fixed size is used so the one-process baseline remains measurable, while still leaving ample work per process at twenty-four processes.) Running time falls steadily as processes are added (Figure 5a). At twelve processes — one per core, the configuration that matches the cluster — the speed-up is about 4.1× (Figure 5b); doubling to twenty-four oversubscribed processes raises it only to 5.4×. The curve bends away from the ideal line as processes are added because the replicated tree build does not shrink with P: it is the sequential fraction of Amdahl's law, and it comes to dominate once the force work per process has become small. Communication, by contrast, stays a few percent of the running time on this shared-memory host (the small gap between the two curves in Figure 5a); on the distributed cluster, where each message crosses the network, it would contribute a larger share. The parallel efficiency (Figure 6) accordingly declines from one as the cost model predicts. One feature is specific to measuring on a single multi-core host: the one-process baseline runs a single core at a higher sustained clock rate and with the full memory bandwidth to itself, so the speed-up it implies at small process counts is a conservative lower bound; this effect does not arise on the

cluster, where each process occupies a separate machine.



(a) Running time, with and without communication.



(b) Speed-up versus the ideal linear line.

Figure 5: Strong scaling at fixed problem size as the process count doubles. The speed-up saturates because the replicated tree build is not divided across processes.

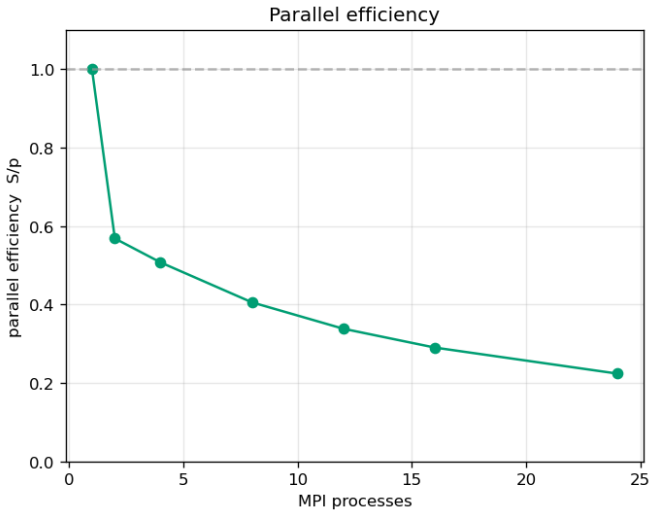
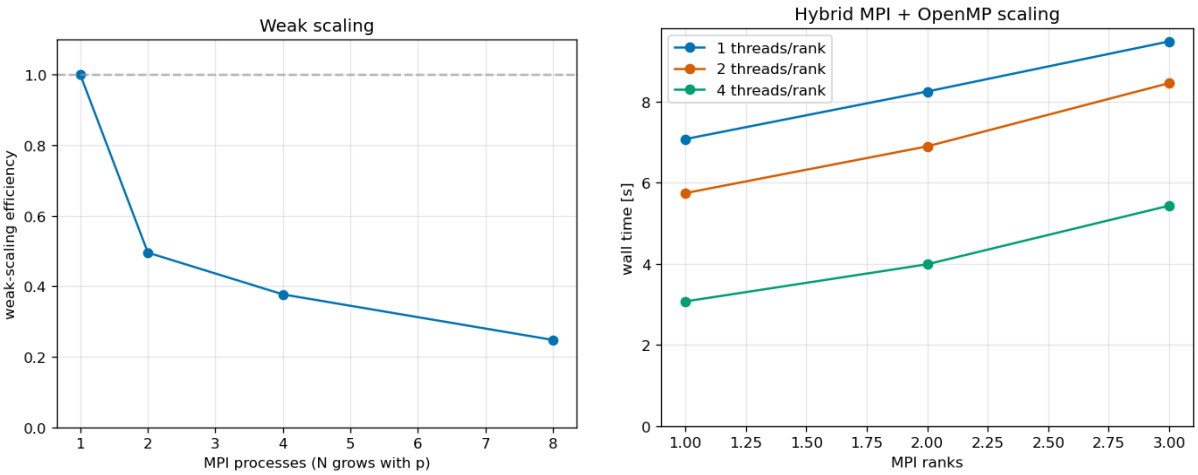


Figure 6: Parallel efficiency $E_P = S_P/P$. The decline with P is the signature of the replicated-build sequential fraction together with the growing all-gather cost.

6.5 Weak scaling and the hybrid layer

Two further experiments round out the picture. In a weak-scaling test, the problem grows in proportion to the process count, so the force work per process is held roughly constant; the running time nonetheless rises and the weak-scaling efficiency falls (Figure 7a). This is the same replicated-build effect seen in the strong-scaling study, viewed from the other direction: the tree build is rebuilt in full on every process and its cost grows with the *total* problem size, so as the problem and the process count grow together the replicated build consumes an ever-larger share. It is a direct, quantitative confirmation that the build, not the communication, is the factor limiting this design. Adding OpenMP threads inside each process reduces the running time further (Figure 7b): with a single process, four threads cut the running time by about 2.3×, and the benefit holds across process counts, confirming that the second, shared-memory level

of parallelism is effective and that the dynamic thread schedule absorbs some of the traversal's irregularity.



(a) Weak-scaling efficiency falls as the problem and process count grow together, because the replicated build grows with the total problem size.

(b) More OpenMP threads per process lower the running time (about 2.3× for four threads at one process), exposing a second level of parallelism.

Figure 7: Weak scaling (left) and the effect of the hybrid shared-memory layer (right).

7 Discussion

The measurements confirm the cost model. The design's strength is that the force evaluation — the expensive phase — is divided cleanly and communicates nothing, so the parallel result is identical to the sequential one and the running time falls steadily as processes are added. Its limitation is equally clear: replicating the tree on every process costs an $\mathcal{O}(N)$ build that is not divided, and this sets the efficiency ceiling. At the scales studied here the build is a modest fraction of each step, but at much larger N or process counts it would dominate, and the replicated storage would also become a constraint. The remedy used in production codes is to distribute the tree itself, partitioning space among processes and exchanging only the boundary information each process needs; this removes the replicated build at the cost of substantially more complex communication [8], and was deliberately left outside our scope.

The load-balance study shows that the one-dimensional block mapping is adequate only because our input is not ordered by position; a position-ordered input exposes the imbalance that an irregular workload can cause, and points to a cyclic distribution or dynamic scheduling as the appropriate granularity adjustment. Finally, the model is two-dimensional for clarity of exposition and demonstration; the extension to three dimensions replaces the quadtree with an octree but leaves the parallel structure unchanged.

8 Conclusion

We implemented a Barnes–Hut N-body simulator, parallelised it for distributed memory with a data decomposition and a single collective exchange per step, and added a hybrid shared-memory layer. The parallel program reproduces the sequential reference exactly and conserves energy, and it achieves a clear speed-up whose ceiling is explained, as the cost model predicts, by the replicated tree build. A per-process timing analysis relates the load balance directly to the mapping and the input ordering. Future work would distribute the tree to remove the sequential build fraction, replace the block mapping with a cyclic or cost-aware distribution to guarantee balance on position-ordered inputs, and extend the model to three dimensions.

Member contributions

All members reviewed the complete program so that any member can explain any part of it, as the project requires.

Member	Primary responsibility	Areas
[member 1]	Sequential method & infrastructure	reference simulator, cluster setup, correctness tests
[member 2]	Distributed-memory parallelism	decomposition, collective communication, integration
[member 3]	Performance study	benchmarking, speed-up and load-balance analysis, figures
[member 4]	Model, visualisation & report	initial conditions, trajectory rendering, report

Table 1: Division of responsibilities. Every component was reviewed by the whole group.

References

[1] J. Barnes and P. Hut. A hierarchical $O(N \log N)$ force-calculation algorithm. *Nature*, 324:446–449, 1986.

[2] M. J. Quinn. *Parallel Programming in C with MPI and OpenMP*. McGraw-Hill, 2004.

[3] A. Grama, A. Gupta, G. Karypis, and V. Kumar. *Introduction to Parallel Computing*. 2nd ed., Addison-Wesley, 2003.

[4] I. Foster. *Designing and Building Parallel Programs*. Addison-Wesley, 1995.

[5] G. M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conf. Proc.*, 30:483–485, 1967.

[6] H. C. Plummer. On the problem of distribution in globular star clusters. *MNRAS*, 71:460–470, 1911.

[7] S. J. Aarseth. *Gravitational N-Body Simulations: Tools and Algorithms*. Cambridge University Press, 2003.

[8] M. S. Warren and J. K. Salmon. A parallel hashed oct-tree N-body algorithm. In *Proc. Supercomputing '93*, pages 12–21, 1993.

A Reproduction

The cluster comprises three machines on one local network, each running the same operating system and MPI library, with passwordless remote access from a designated master and the program staged at the same path on every machine. After building the three executables — sequential, distributed-memory and hybrid — the correctness checks, the running-time, load-balance and speed-up studies, and the figures are each produced by a single command, with the number of processes set equal to the number of cores and the problem size set as described in Section 5. The exact commands and parameters are provided with the source.